

Math Foundations, Vectors & Matrices

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

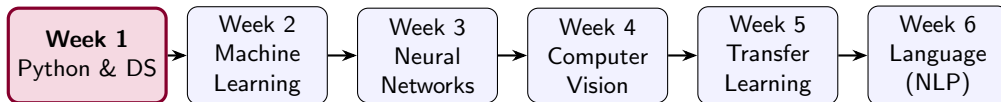


موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 1, Day 02, Lecture

20 September 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. Math Foundations
4. Scalars and Vectors
5. The Dot Product
6. Norm and Distance
7. Matrices
8. Matrix Multiplication
9. NumPy Arrays
10. Olympiad Practice
11. Limitations and What Comes Next
12. Summary
13. References



Today: Day 2 of 42

- ▶ **Yesterday:** values, lists, loops, functions, a first class. You can add up a list and classify a flower with an `if`.
- ▶ **Today:** the math of lists of numbers (vectors) and tables of numbers (matrices), and NumPy, the library that computes with them.
- ▶ **Why now:** every model from Week 2 on is vectors and matrices multiplied together. Today you learn to read that language.

Yesterday's Lab 2 used one measurement (petal length) and a loop for every calculation.

```
total = 0
for row in rows:                # average petal length, by hand
    total = total + row["PetalLengthCm"]
average = total / len(rows)

def predict(row):                # one number decides
    if row["PetalLengthCm"] < 2.5:
        return "Iris-setosa"
    ...
```

It worked: 95% accuracy. But every new calculation is a new loop, and we used one of the four measurements.

The problem

A flower is **4** numbers. A 28×28 image is **784** numbers. A sentence in Week 6 becomes **768** numbers. To compare two flowers you need all four measurements at once; a loop per measurement per operation means code that is long, slow and easy to get wrong.

- ▶ **Scale:** the Iris table is $150 \times 4 = 600$ numbers; the Adult table is $32,561 \times 14 \approx 456,000$.
- ▶ **Speed:** yesterday's figure: a plain loop was 25 to 50 times slower than NumPy on a million numbers.
- ▶ **Errors:** "add these two rows" as a loop over indices is where off-by-one mistakes live.

In one sentence

Treat a list of numbers as **one object**, a **vector**, and a table of numbers as one object, a **matrix**. Math gives us a few operations on them (add, scale, dot product, multiply); NumPy runs those operations in one line, fast.

Where you will meet it

In Lab 2 you describe each flower by all four measurements as a vector, compute the “centre” of each species, and classify by the nearest centre. No hand-picked threshold: the math finds the boundary. Week 2 calls this a classifier.

By the end of this session, you will be able to:

- ▶ **Read** sum notation and compute a mean and a standard deviation by hand.
- ▶ **Explain** what a vector and a matrix are, and write their shapes.
- ▶ **Compute** vector addition, scalar multiplication, the dot product, the norm and the distance between two vectors, with numbers.
- ▶ **Compute** a transpose, a matrix-vector product and a matrix-matrix product, and check the shape rule.
- ▶ **Use** NumPy arrays: create, index, slice, and do the operations above in one line.
- ▶ **Recognise** a matrix product inside an AI model and a shape error in a traceback.

Math Foundations: **Functions, Sums and Averages**

A Function Is a Rule From Input to Output

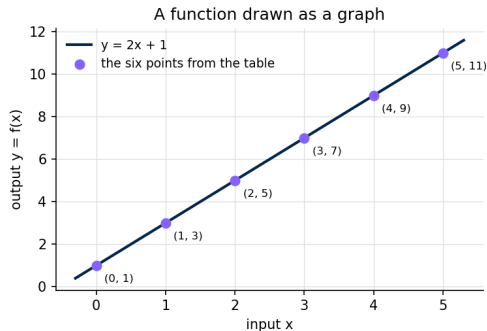
Definition

A **function** f turns an input x into one output $y = f(x)$. Same input, same output, every time.

$$f(x) = 2x + 1$$

x	0	1	2	3	4	5
$f(x)$	1	3	5	7	9	11

Worked: $f(3) = 2 \cdot 3 + 1 = 7$. The same rule as yesterday's `def f(x): return 2*x + 1`.



Source: `scripts/day02_math_figures.py`.

$$\sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$$

- ▶ $\sum$ (“sigma”): add up. i : the counter. $i = 1$ below: where it starts. n above: where it stops. x_i : the i -th item.
- ▶ **Worked example:** scores $x = (78, 92, 85, 64, 91)$, $n = 5$:
 $\sum_{i=1}^5 x_i = 78 + 92 + 85 + 64 + 91 = 410$.
- ▶ **With a rule inside:** $\sum_{i=1}^3 i^2 = 1 + 4 + 9 = 14$.

It is yesterday’s accumulator loop

`total = 0; for x in xs: total += x` is exactly $\sum x_i$. When you see $\sum$, think “a for loop that adds”.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- ▶ $\bar{x}$ (“x bar”): the mean, the centre of the numbers. n : how many numbers.
- ▶ **Worked example:** $\bar{x} = \frac{410}{5} = 82$.
- ▶ The mean is the first thing you compute about any column of any dataset, and the centre of each species in Lab 2.

Not to be confused with

The **median** is the middle value when sorted (64, 78, **85**, 91, 92: median 85). One huge score moves the mean a lot and the median little.

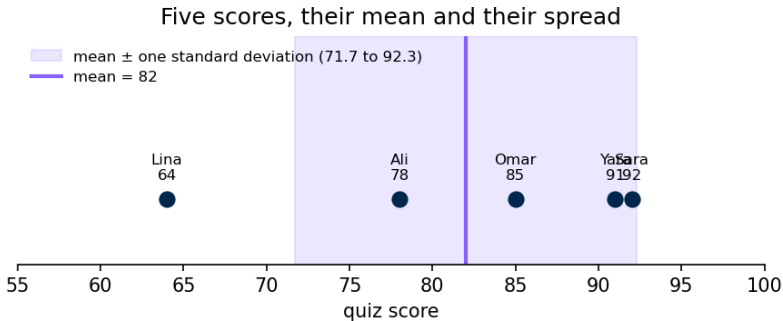
Variance and Standard Deviation: How Spread Out?

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \qquad \sigma = \sqrt{\sigma^2}$$

σ^2 : the **variance**, the average squared distance from the mean. σ (“sigma”): the **standard deviation**, in the same units as the data.

x_i	78	92	85	64	91	sum
$x_i - \bar{x}$	-4	10	3	-18	9	0
$(x_i - \bar{x})^2$	16	100	9	324	81	530

$\sigma^2 = 530/5 = 106$, so $\sigma = \sqrt{106} \approx 10.3$. Most scores sit within about 10 points of the mean 82.



Source: `scripts/day02.math.figures.py`, the five quiz scores.

Three of five scores fall inside one standard deviation of the mean. Lina and Sara are the outliers. Standard deviation is how Week 2 “scales” features so that centimetres and grams can be compared.

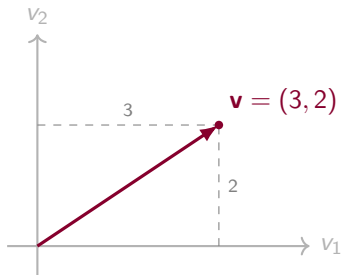
Math Foundations: **Scalars and Vectors**

Definitions

A **scalar** is a single number: $a = 3$. A **vector** is an ordered list of numbers: $\mathbf{v} = (1, 2, 3)$. The number of entries is its **dimension**: $\mathbf{v}$ has dimension 3, written $\mathbf{v} \in \mathbb{R}^3$.

- ▶ Written in bold: $\mathbf{v}$. Its i -th entry is v_i : here $v_1 = 1$, $v_2 = 2$, $v_3 = 3$ (math counts from 1, Python from 0).
- ▶ **One flower is a vector**: $\mathbf{x} = (5.1, 3.5, 1.4, 0.2)$, four measurements in a fixed order, $\mathbf{x} \in \mathbb{R}^4$.
- ▶ **Not to be confused with**: a Python list can mix types and change length. A vector is numbers only, fixed length.

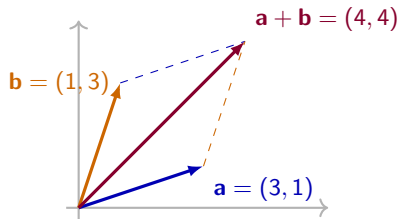
A Vector Is Also an Arrow



- ▶ A 2-dimensional vector is a **point** (3 to the right, 2 up) and an **arrow** from the origin to it.
- ▶ In 3 dimensions you can still picture it. In 4 (a flower) or 784 (an image) you cannot, but every formula stays the same.
- ▶ Tomorrow's scatter plots draw 2-dimensional vectors: each dot is one row of the table.

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$$

- ▶ Add entry by entry. Both vectors must have the **same dimension**.
- ▶ **Worked:** $(1, 2, 3) + (4, 5, 6) = (5, 7, 9)$.
- ▶ Subtraction is the same, entry by entry:
 $(4, 5, 6) - (1, 2, 3) = (3, 3, 3)$.
- ▶ $(1, 2) + (1, 2, 3)$ is **undefined**: dimensions 2 and 3.



Picture: put $\mathbf{b}$'s tail on $\mathbf{a}$'s head; the sum is the arrow to the far corner.

$$c \mathbf{a} = (c a_1, c a_2, \dots, c a_n)$$

- ▶ Multiply every entry by the same number c . The arrow keeps its direction and changes its length (flips if $c < 0$).
- ▶ **Worked:** $2 \cdot (1, 2, 3) = (2, 4, 6)$; $\frac{1}{2} \cdot (4, 6) = (2, 3)$; $-1 \cdot (3, 1) = (-3, -1)$.
- ▶ **Where you meet it:** the mean of n vectors is their sum scaled by $\frac{1}{n}$. In Lab 2 the centre of a species is $\frac{1}{50} \sum \mathbf{x}_i$ over its 50 flowers, a vector of four averages.

```
a = [1, 2, 3]
b = [4, 5, 6]
a + b                                # [1, 2, 3, 4, 5, 6]  lists glue
[x + y for x, y in zip(a, b)]       # [5, 7, 9]    vector addition
[2 * x for x in a]                  # [2, 4, 6]    scalar multiplication
```

Lists are not vectors

+ on lists means “join”. $2 * a$ repeats the list. Every vector operation needs a loop. This is the pain NumPy removes at the end of today.

Math Foundations: **The Dot Product**

Definition

The **dot product** of two vectors of the same dimension multiplies them entry by entry and adds the results. The answer is one number (a scalar).

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

- ▶ **Worked:** $(1, 2, 3) \cdot (4, 5, 6) = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32$.
- ▶ Two vectors in, one number out. Same dimension required.
- ▶ **Not to be confused with:** entry-by-entry multiplication $(1, 2, 3) * (4, 5, 6) = (4, 10, 18)$ gives a vector; the dot product adds those up.

A quiz has three rounds with different weights: round 1 counts 20%, round 2 30%, round 3 50%.

$$\text{final} = \mathbf{w} \cdot \mathbf{x} = (0.2, 0.3, 0.5) \cdot (92, 88, 96) = 18.4 + 26.4 + 48.0 = 92.8$$

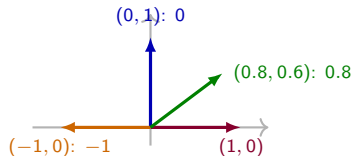
- ▶ $\mathbf{x}$: the scores. $\mathbf{w}$: the weights, how much each score matters.
- ▶ Every entry of $\mathbf{x}$ is scaled by its weight, then everything is added. That is all a dot product does.

Where you will meet it

Week 2's linear model is $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$: weights times measurements plus a constant. Week 3's neuron is the same dot product followed by a switch. Learning means finding good $\mathbf{w}$.

The Dot Product Measures Alignment

a	b	$a \cdot b$	
(1, 0)	(1, 0)	1	same direction
(1, 0)	(0, 1)	0	at right angles
(1, 0)	(-1, 0)	-1	opposite
(3, 1)	(1, 3)	6	roughly the same way



Positive: pointing the same way. Zero: unrelated. Negative: opposite. This is why “similarity” in Week 5 (matching an image to a caption) is a dot product.

```
a = [1, 2, 3]
b = [4, 5, 6]
total = 0
for x, y in zip(a, b):           # walk both vectors together
    total = total + x * y        # multiply the pair, add to the sum
print(total)                     # 32
```

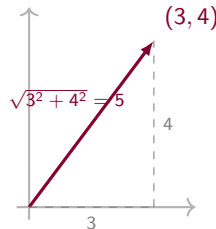
The accumulator pattern with a product inside. Two lines shorter as a function; one line in NumPy: `np.dot(a, b)`.

Math Foundations: **Norm and Distance**

The Norm: How Long Is a Vector?

$$\|\mathbf{a}\| = \sqrt{a_1^2 + a_2^2 + \cdots + a_n^2} = \sqrt{\mathbf{a} \cdot \mathbf{a}}$$

- ▶ $\|\mathbf{a}\|$: the **norm** (length) of the arrow. Square each entry, add, take the square root.
- ▶ **Worked:** $\|(3, 4)\| = \sqrt{9 + 16} = \sqrt{25} = 5$.
- ▶ In 2 dimensions this is Pythagoras. In 4 or 784 dimensions the formula is identical.
- ▶ $\|(1, 2, 2)\| = \sqrt{1 + 4 + 4} = 3$.

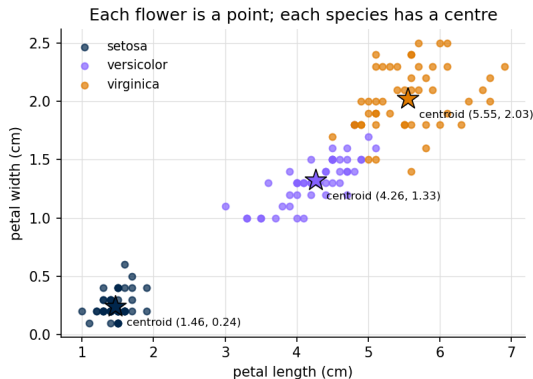


$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\| = \sqrt{(a_1 - b_1)^2 + \cdots + (a_n - b_n)^2}$$

- ▶ Subtract entry by entry, then take the norm of the result. “How far apart are these two points?”
- ▶ **Worked:** $\mathbf{a} = (1, 1)$, $\mathbf{b} = (4, 5)$: $\mathbf{a} - \mathbf{b} = (-3, -4)$, $d = \sqrt{9 + 16} = 5$.
- ▶ **Two flowers:** *setosa* (5.1, 3.5, 1.4, 0.2) and *virginica* (6.3, 3.3, 6.0, 2.5): difference $(-1.2, 0.2, -4.6, -2.3)$, squares $1.44 + 0.04 + 21.16 + 5.29 = 27.93$, $d \approx 5.28$.

Skipped on purpose

Other distances (Manhattan, cosine) and why scale matters: Day 05 and Week 2. Today: one distance, the Euclidean one above.



Source: `scripts/day02.math-figures.py`, Iris data set (Kaggle uciml/iris).

1. Each flower is a point (a vector).
2. Each species has a **centroid**: the mean vector of its flowers (a star).
3. A new flower gets the species whose centroid is **closest** (smallest distance).

No threshold chosen by hand. This is Lab 2, and the ancestor of Week 2's nearest-neighbour method.

```
import math

def norm(a):
    total = 0
    for x in a:
        total = total + x * x
    return math.sqrt(total)

def distance(a, b):
    diff = [x - y for x, y in zip(a, b)]    # a - b, entry by entry
    return norm(diff)

print(norm([3, 4]), distance([1, 1], [4, 5]))    # 5.0 5.0
```

Math Foundations: **Matrices**

Definition

A **matrix** is a rectangular grid of numbers with m rows and n columns. Its **shape** is $m \times n$ (rows first).

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \text{shape } 2 \times 3$$

- ▶ Written in bold capitals: **A**.
- ▶ Entry a_{ij} : row i , column j . Here $a_{12} = 2$, $a_{23} = 6$.
- ▶ Row 2 is the vector (4, 5, 6); column 3 is (3, 6).
- ▶ **A dataset is a matrix:** Iris is 150×4 , one row per flower, one column per measurement. Row i is flower i 's vector.
- ▶ **An image is a matrix:** a 28×28 grey picture is 784 brightness numbers (Day 05 and Week 4).
- ▶ A vector is a matrix with one column ($n \times 1$).

Same as vectors: entry by entry, same shape required.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 11 & 22 \\ 33 & 44 \end{bmatrix} \qquad 2 \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

- ▶ $(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij}$ and $(c\mathbf{A})_{ij} = c a_{ij}$.
- ▶ A 2×2 plus a 2×3 is undefined.
- ▶ **Where you meet it:** Week 3 updates every weight matrix as $\mathbf{W} - 0.01 \cdot \mathbf{G}$: a scaled matrix subtracted from another.

The Transpose: Rows Become Columns

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} (2 \times 3) \quad \mathbf{A}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} (3 \times 2)$$

- ▶ $(\mathbf{A}^T)_{ij} = a_{ji}$: flip over the diagonal. Row 1 of $\mathbf{A}$, $(1, 2, 3)$, is column 1 of $\mathbf{A}^T$.
- ▶ Shape $m \times n$ becomes $n \times m$.
- ▶ **Where you meet it:** the Iris matrix is flowers $\times$ measurements. Its transpose is measurements $\times$ flowers: row 3 of $\mathbf{X}^T$ is all 150 petal lengths, ready for a mean.

```
A = [[1, 2, 3],  
      [4, 5, 6]]  
A[1][2]  
# a list of rows  
# 6 row 1, column 2 (Python  
# counts from 0)  
len(A), len(A[0])  
# (2, 3) rows, columns  
  
AT = [[A[i][j] for i in range(2)] for j in range(3)] # transpose  
# [[1, 4], [2, 5], [3, 6]]
```

- ▶ Math says $a_{12} = 2$ (row 1, column 2, counting from 1). Python says `A[0][1]`. Keep the two in your head.
- ▶ Even the transpose needs a double loop. Column 2 of `A` has no direct name: you must loop over rows.

Math Foundations: **Matrix Multiplication**

Definition

$\mathbf{A}\mathbf{v}$ takes the dot product of **each row** of $\mathbf{A}$ with $\mathbf{v}$. An $m \times n$ matrix times an n -vector gives an m -vector.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 1 \\ 3 \cdot 1 + 4 \cdot 1 \\ 5 \cdot 1 + 6 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 7 \\ 11 \end{bmatrix}$$

- ▶ Shape: $(3 \times 2)(2 \times 1) = 3 \times 1$. The matrix has as many columns as the vector has entries.
- ▶ **Where you meet it:** the quiz weights (0.2, 0.3, 0.5) applied to every student at once: scores matrix (6×3) times weights (3-vector) = six finals in one product.

Definition

$\mathbf{C} = \mathbf{AB}$: entry c_{ij} is the dot product of **row i of $\mathbf{A}$** with **column j of $\mathbf{B}$** .

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Entry	Row of $\mathbf{A}$	Column of $\mathbf{B}$	Dot product
c_{11}	(1, 2)	(5, 7)	$1 \cdot 5 + 2 \cdot 7 = 19$
c_{12}	(1, 2)	(6, 8)	$1 \cdot 6 + 2 \cdot 8 = 22$
c_{21}	(3, 4)	(5, 7)	$3 \cdot 5 + 4 \cdot 7 = 43$
c_{22}	(3, 4)	(6, 8)	$3 \cdot 6 + 4 \cdot 8 = 50$

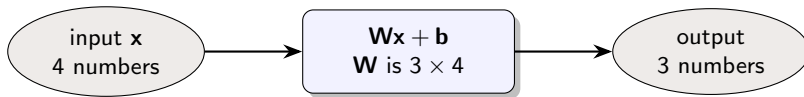
$$(m \times \underbrace{n}_{\text{must match}})(n \times p) = m \times p$$

- ▶ The **inner** numbers must be equal: columns of the left matrix = rows of the right one.
- ▶ The **outer** numbers give the answer's shape.
- ▶ **Worked:** $(150 \times 4)(4 \times 3) = 150 \times 3$: every flower times three columns of weights gives three scores per flower.
- ▶ $(2 \times 3)(2 \times 3)$ is **undefined**: $3 \neq 2$. Order matters: **AB** and **BA** are different, and often only one exists.

The most common bug from Week 3 on

shapes $(150, 4)$ and $(3, 4)$ not aligned: the inner numbers differ. Fix: transpose one side, $(150 \times 4)(4 \times 3)$.

Where You Will Meet It: A Layer Is a Matrix Product



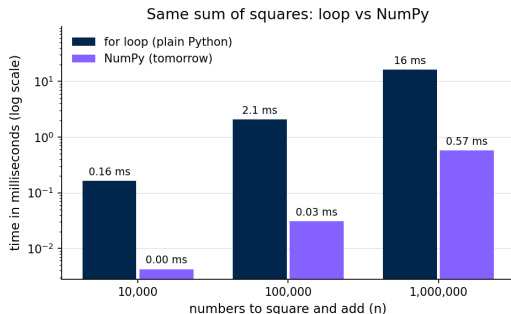
- ▶ Week 2: one row of weights, $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$.
- ▶ Week 3: several rows at once, $\mathbf{W}\mathbf{x} + \mathbf{b}$; a network is a chain of these.
- ▶ Week 6: attention scores are $\mathbf{Q}\mathbf{K}^\top$, one matrix product. The formula never changes; only the shapes grow.

```
def matvec(A, v):                                # A: list of rows, v: list
    result = []
    for row in A:                                # one dot product per row
        total = 0
        for a, x in zip(row, v):
            total = total + a * x
        result.append(total)
    return result
```

```
matvec([[1, 2], [3, 4], [5, 6]], [1, 1])        # [3, 7, 11]
```

Three nested ideas: a loop over rows, a dot product inside, an append. Matrix-matrix is one more loop over columns. In NumPy: $A @ v$.

Math Foundations: **NumPy** Arrays



Source: `scripts/day01_loop.vs_numpy.py`, measured on the instructor's laptop.

Definition

A NumPy **array** is a vector or matrix stored as one object: fixed shape, one number type, and every math operation built in.

- ▶ Every loop from today becomes one line.
- ▶ 25 to 50 times faster on a million numbers, and more on big matrices.
- ▶ pandas (tomorrow) and PyTorch (Week 3) are built on the same idea.

```
import numpy as np
a = np.array([1, 2, 3])
A = np.array([[1, 2, 3],
              [4, 5, 6]])
a.shape, A.shape
A.dtype
np.zeros(4)
np.arange(5)
```

```
# a vector from a list
# a matrix from a list of rows
# (3,) (2, 3) rows, columns
# int64 one type for all
# [0. 0. 0. 0.]
# [0 1 2 3 4] like range
```

- ▶ shape is a tuple (yesterday's "list that cannot change"). (2, 3) means 2×3 : math and NumPy agree.
- ▶ An array with a decimal anywhere becomes all floats: `np.array([1, 2.5]).dtype` is `float64`.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

A[1, 2]          # 6                row 1, column 2 (one pair of brackets)
A[0]             # [1 2 3]          row 0
A[:, 2]          # [3 6]           column 2: all rows, column 2
A[:, 1:3]        # [[2 3] [5 6]]   all rows, columns 1 and 2
a = np.array([78, 92, 85, 64, 91])
a[1:4]           # [92 85 64]      slicing works like lists
```

- ▶ `A[row, column]`: the math notation a_{ij} , counting from 0.
- ▶ `:` alone means “all of them”. `A[:, 2]` is the column no plain list could name.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
a + b          # [5 7 9]          vector addition
a - b          # [-3 -3 -3]
2 * a          # [2 4 6]          scalar multiplication
a * b          # [4 10 18]        entry by entry, NOT the dot product
np.dot(a, b)   # 32               the dot product
a @ b          # 32               same thing, the @ operator
np.linalg.norm(b - a) # 5.196    distance between a and b
```

* versus @

$a * b$ multiplies entry by entry and gives a vector. $a @ b$ is the dot product and gives a number. Mixing them up is silent: no error, wrong answer.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
v = np.array([1, 1])
A @ v          # [3 7]                matrix times vector
A @ B          # [[19 22] [43 50]]    matrix times matrix
A.T            # [[1 3] [2 4]]        transpose
A + B, 2 * A   # entry by entry, as with vectors
C = np.array([[1, 2, 3]]) # shape (1, 3)
C @ B          # ValueError: shapes (1,3), (2,2) not aligned
```

The last line is the shape rule as an error message: inner numbers 3 and 2 differ. Read the shapes in the message, then transpose or fix the data.

```
scores = np.array([78, 92, 85, 64, 91])
scores.sum()           # 410
scores.mean()          # 82.0
scores.std()           # 10.296  this morning's sigma
scores.min(), scores.max() # 64 91
X = np.array([[5.1, 3.5, 1.4, 0.2],
              [6.3, 3.3, 6.0, 2.5]])
X[:, 2].mean()         # 3.7  mean of column 2 over the rows
```

Skipped on purpose

`X.mean(axis=0)` for all columns at once, broadcasting (`X - X.mean()`), reshape, and boolean masks (`X[y == "setosa"]`). Day 05 covers them. Today: one column at a time with `X[:, j]`, which is all Lab 2 needs.

Math Foundations: **Olympiad Practice**

Task

Write a function `matrix_dot_vector(a, b)` where `a` is a matrix given as a list of rows and `b` is a vector given as a list. Return the product **`ab`** as a list. If the shapes do not allow the product, return `-1`.

- ▶ Example input: `a = [[1, 2], [2, 4]]`, `b = [1, 2]`. Expected output: `[5, 10]`.
- ▶ Example input: `a = [[1, 2], [2, 4]]`, `b = [1, 2, 3]`. Expected output: `-1`.
- ▶ Scoring: hidden test cases, all must pass. Plain Python only (no NumPy in the graded function).

Adapted from: Deep-ML Open Problem Bank, problem 1 "Matrix-Vector Dot Product" (easy).

1. **Recognise:** matrix times vector is one dot product per row. The dot product is the accumulator loop with `zip`. The shape rule says: number of columns of `a` must equal `len(b)`.
2. **Plan:**
 - Check the shape first: if `len(a[0]) != len(b)`, return `-1`.
 - For each row: $\text{total} = \sum \text{row entry} \times \text{vector entry}$; append to the result.
3. **Check:** by hand, row $(1, 2) \cdot (1, 2) = 1 + 4 = 5$ and row $(2, 4) \cdot (1, 2) = 2 + 8 = 10$. Then compare with `np.array(a) @ np.array(b)` on a few random inputs before submitting.

```
def matrix_dot_vector(a, b):  
    if len(a[0]) != len(b):          # columns of a must equal len(b)  
        return -1  
    result = []  
    for row in a:  
        total = 0  
        for x, y in zip(row, b):  
            total = total + x * y  
        result.append(total)  
    return result  
  
matrix_dot_vector([[1, 2], [2, 4]], [1, 2])      # [5, 10]
```

Math Foundations: **Limitations & What Comes Next**

- ▶ **Arrays hold one type of number.** Names, species labels and dates do not fit. Tomorrow's pandas table holds mixed columns and gives them names.
- ▶ **Scale is not free.** Distance in Lab 2 treats 1 cm of sepal length like 1 cm of petal width. When units differ (cm and grams), one column dominates. Week 2 fixes this with scaling (the standard deviation from this morning).
- ▶ **One column at a time is slow to write.** `axis`, broadcasting and masks (Day 05) do all columns at once.

The three mistakes to watch for

- ▶ **Shape mismatch:** read the two shapes in the error; the inner numbers must match; transpose if needed.
- ▶ *** versus @:** entry-wise product versus dot product. No error, wrong number.
- ▶ **Math counts from 1, Python from 0:** a_{12} is `A[0, 1]`.

- ▶ **Tomorrow, Day 03: EDA, Pandas, Visualization.** The Iris matrix becomes a DataFrame with named columns; means, counts and plots in one call each; and cleaning dirty data.
- ▶ **Wednesday, Day 05: NumPy & Linear Algebra (self-study).** Broadcasting, axis, reshape, boolean masks, an image as a matrix, and matrix multiplication step by step.
- ▶ **Week 2:** $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ becomes a model, and gradient descent finds $\mathbf{w}$.
- ▶ **Friday:** the Coursera exam and the Week 1 lab exercise.

Math Foundations: **Summary**

- ▶ $\sum$ is a **loop that adds**; the **mean** is the sum over n ; the **standard deviation** is the typical distance from the mean.
- ▶ A **vector** is a fixed list of numbers (one flower); a **matrix** is a grid with shape rows $\times$ columns (one dataset).
- ▶ Vectors **add** and **scale** entry by entry; the **dot product** multiplies and adds to one number (a weighted sum); the **norm** is the length and the **distance** is the norm of the difference.
- ▶ The **transpose** flips rows and columns; a **matrix product** is a dot product per row and column, and needs the inner shapes to match.
- ▶ **NumPy arrays** do all of this in one line: `+`, `*`, `@`, `.T`, `.mean()`, `A[:, j]`.
- ▶ A **shape error** names the two shapes; $\mathbf{W}\mathbf{x} + \mathbf{b}$ is the model you will meet in every later week.

Math Foundations: **References & Resources**

- [1] Open-Deep-ML (2025). *DML-OpenProblem*, problem 1: Matrix-Vector Dot Product. The practice task is adapted from it.
<https://github.com/Open-Deep-ML/DML-OpenProblem>
- [2] Harris, C. R. et al. (2020). Array programming with NumPy. *Nature* 585, 357–362. NumPy user guide: https://numpy.org/doc/stable/user/absolute_beginners.html
- [3] Fisher, R. A. (1936). *Iris* data set, Kaggle copy uciml/iris, used for the centroid figure and Lab 2. <https://www.kaggle.com/datasets/uciml/iris>
- [4] Strang, G. (2016). *Introduction to Linear Algebra*, 5th ed. Wellesley-Cambridge Press. Chapters 1 and 2 cover today's vectors and matrices.
- [5] Altwaim, S. (2026). *Day 1: Math & Code Foundations*, KAUST Academy & Mawhiba IOAI 2026 training. Dot product, matrix product and shape-rule examples adapted from it.